



BasketBot: Smart Basketball Hoop

Submitted by: Biel Cruz, Ferran Soler, David Sánchez, and Èric Vila
Project: Integrated Project II

Summary

BasketBot is a smart basketball hoop designed to automate score tracking, time control, and turn management in informal games. The primary objective of this mechatronic system is to resolve typical on-court disputes, allowing players to focus exclusively on the game. To achieve this, the project integrates embedded hardware, electronic scoreboards, and a web control interface.

At the hardware level, the hoop features an automated lifting mechanism driven by a NEMA 17 stepper motor with an integrated lead screw, which allows the height of the structure to be adjusted remotely for different types of players. The core of the system is an ESP32 microcontroller that acts as a local Wi-Fi access point and manages peripheral devices. This includes a TMC2209 driver for smooth motor control, a dual infrared sensor (TSOP58438 receivers) for precise and interference-free ball detection, and TM1637 displays alongside LED indicators for instantaneous visual feedback.

The software has been designed to operate asynchronously, ensuring that basket detection and motor movement do not interrupt network communication. Through the locally hosted web application, users can select from four different game modes: Free Practice, The 21, 1 vs 1, and Shootout. During functional testing, communication between the board and the web app demonstrated unnoticeable latency, and the detection accuracy reached 87%, while also overcoming initial mechanical obstacles by stabilizing the lifting structure.

Table of Contents

1. Introduction / State of the art – Benchmarking.....	3
2. Objectives.....	4
3. Development of the project.....	5
3.1. Analysis of the problem and possible solutions.....	5
3.2. Description of the design.....	6
3.2.1. Mechanical design.....	6
Fixed Structural Components.....	6
Mobile Structural Components.....	7
3.2.2. Electrical design.....	9
Connection Mapping and Pin Assignment.....	9
Control Architecture (The “Brain”).....	10
Motor NEMA 17 stepper motor with integrated T8 lead screw.....	11
Driver: TMC2209.....	12
Basic Operation of the Motor and Driver.....	12
Display Connections.....	13
Feedback Indicators (RGB / Simple LEDs).....	14
Interconnection and Integration with Sensors.....	15
3.2.3. Control and software.....	16
Game and Team Validation Flow Logic.....	16
Game Modes.....	16
Motor Control.....	19
Hoop Sensors.....	20
Phase 1: Testing and calibration program.....	20
Phase 2: Final Code.....	20
Displays.....	21
Web Application and Wi-Fi Communication.....	22
3.3. Functional testing of the prototype.....	23
4. Cost of the project.....	25
4.1. Engineering Man-Hours.....	25
4.2. Material Budget.....	26
Table 3 Material Budget.....	26
4.3. Total Project Budget.....	26
5. Conclusions.....	27
6. Bibliography.....	28

1. Introduction / State of the art – Benchmarking

Basketball is a global sport, played at both professional and recreational levels. In informal settings—such as schoolyards, parks, or sports centers—matches are typically self-managed by the players themselves. This lack of an external refereeing figure delegates the tasks of tracking the score, managing possession turns, and enforcing the rules of the game to the participants.

The dynamic nature of the game often leads to confusion, involuntary errors, or, occasionally, dishonest behavior that diminishes the sporting experience. The primary motivation of this project is the creation of a smart basketball hoop capable of automating score recording, turn management, and timing.

The idea wasn't born in a lab, but on the court itself. Anyone who has ever played a pickup game with friends knows the drill: you're winning, the game gets intense, and suddenly, nobody can agree on the score. "Was it 10–8 or 11–9?" That minor doubt sparks arguments and breaks the rhythm of the game.

We are building this project because we want players to forget about the numbers and focus on the ball. We want to use technology to resolve those conflicts and keep the game as clean and fluid as possible. Ultimately, the goal is for the hoop to work for us, not the other way around.

2. Objectives

Following the baseline criteria established in the initial project milestones, this system bridges the gap between digital score-tracking web frameworks and automated physical machinery to solve the disputes and tracking errors typical of self-managed, informal basketball games. The main objective of this project is to design, program, and build "BasketBot"—a fully functional, self-contained smart basketball hoop system that integrates embedded hardware and electronic scoreboards with a local WiFi web application to manage games, scoring, timing, and automated mechanical height positioning. Operated by an ESP32 microcontroller acting as a local Access Point, the system coordinates multi-display time and scoreboards alongside a responsive mobile interface to select interactive game modes and dynamically configure match parameters.

To achieve this, the specific objectives focus on delivering robust mechatronic automation, high-integrity electronics, and reliable software logic. The system implements automated mechanical height positioning spanning a linear travel, driven by a NEMA 17 stepper motor and regulated via a physical microswitch homing sequence. On the data layer, precise score registration is guaranteed by utilizing a specialized edge-detection algorithm (HIGH-to-LOW transition logic) with strict software cooldown timers to eliminate multi-count sensor ghosting. Power distribution is optimized using an LM2596 DC-DC buck converter to step down a stable 12V main source into a clean 5V line connected directly to the ESP32 VIN pin, protecting sensitive digital control logic from inductive motor loads. Finally, the entire modular software architecture and interactive game routines are structured, version-controlled, and documented collaboratively using a Git repository on GitHub.

3. Development of the project

In order to achieve the final prototype and fulfill all the proposed objectives, it is essential to divide the project development into several core areas. On one hand, we find the tasks focused on mechanical design, such as numerical calculations, 3D modeling, and lifting mechanisms. On the other hand, the project focuses on the electronic and control systems, which encompass the electrical schematics, sensor integration, and mobile app connectivity. Finally, all aspects regarding software programming and system firmware are thoroughly developed.

3.1. Analysis of the problem and possible solutions

To achieve a variable-height basketball hoop that syncs data with a mobile application, several architectural solutions were considered:

- **Mechanical Lifting Mechanism:** We evaluated several options for the basket-lifting system, including a gear-based mechanism. But in the end, for the sake of simplicity and control, we opted for a motorized lever mechanism driven by a stepper motor.

In this system, the motor drives a rigid rod connected directly to the backboard mount. This creates an inverted lever system: when the motor pushes the rod upward, the basketball hoop moves downward, and vice versa.

- **Electronic Score Tracking:** We analyzed using ultrasonic sensors, infrared (IR) break-beam sensors, or a camera system to detect when a basket is scored. IR/Ultrasonic sensors were selected due to their reliability and low computational requirements.
- **Connectivity:** Wi-Fi in Access Point (AP) mode was chosen to facilitate direct, low-latency communication with the web application hosted on the ESP32. By establishing its own local network, the microcontroller allows users to access the control interface directly via a web browser using its local IP address, completely eliminating the need for an external network or internet connection.

3.2. Description of the design

The BasketBot is a mechatronic system defined by the following core characteristics:

- **Dimensions & Weight:** 65 cm long by 35 cm wide. Weighing 6 kg.
- **Degrees of Freedom:** One vertical degree of freedom (translation for height adjustment).
- **Power Supply:** The system is powered from a single external 12V DC power supply. A DC-DC step-down regulator (LM2596) converts this voltage to 5V, which powers the two TM1637 displays (scoreboard and timer). The remaining components — ESP32, TSOP4838, ultra sonics sensor, LEDs, push buttons, and limit switches — operate at 3.3V, supplied by the ESP32's internal regulator. The TMC2209 driver and the NEMA 17 motor are powered directly from the 12V input. The 330 Ω and 220 Ω resistors limit the current of the LEDs, protecting them.
- **Microcontroller & Language:** The system is controlled by an ESP32 microcontroller, programmed in Arduino using the Arduino IDE.

3.2.1. Mechanical design

The mechanical design of the system was conceived to fulfil two main objectives: to provide a rigid and robust structure capable of withstanding impacts and movements, and to implement a reliable vertical lifting mechanism. The design is clearly divided into fixed structural components and mobile structural components.

Fixed Structural Components

Rim:

It is a simple design, engineered with enough strength to withstand ball impacts corresponding to its size, yet retaining the flexibility that characterizes a basketball hoop rim. It also features four holes on the back section to ensure it can be easily and securely attached to the backboard.

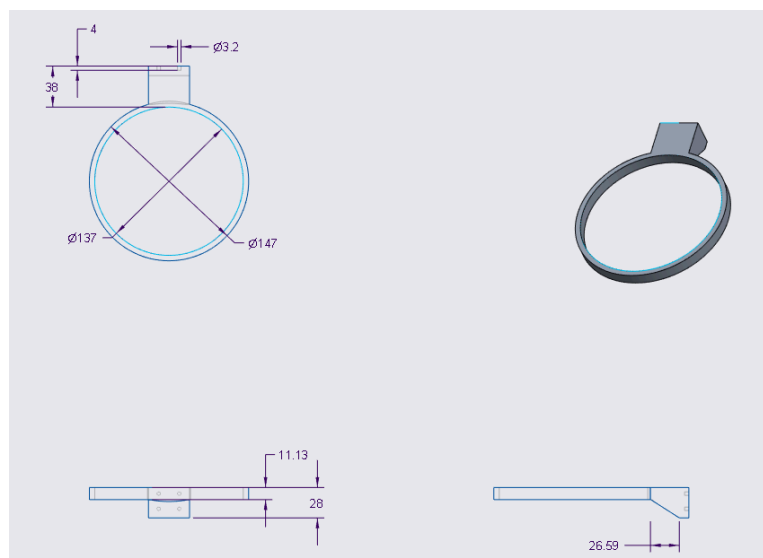


Figure 1 Rim

Main Structure:

The main body of the structure is constructed from wood, which provides a solid and stable frame for the system. To achieve the exact required height, a custom plastic base was integrated at the bottom for precise adjustment. Additionally, the assembly includes a counterweight system designed to balance the structure's weight, ensuring that the basket remains stable and does not tilt or fall forward during operation.

Mobile Structural Components

Support Arms and Sliding Carriages:

The mechanism that moves and supports the board operates like a dual-arm parallel system. It consists of two pairs of parts stacked one above the other. This gives the board four points of support at all times, preventing it from swaying or tilting forward as it rises and falls.

On one side, we have the support arms, which are two identical pieces attached to the back of the panel. On the other side, these arms connect to the following square pieces. These are hollow pieces that are fixed to the basket body. One of them has a pin or joint on the back connected directly to the driveshaft. In this way, it receives the force from the engine directly and pulls the entire assembly.

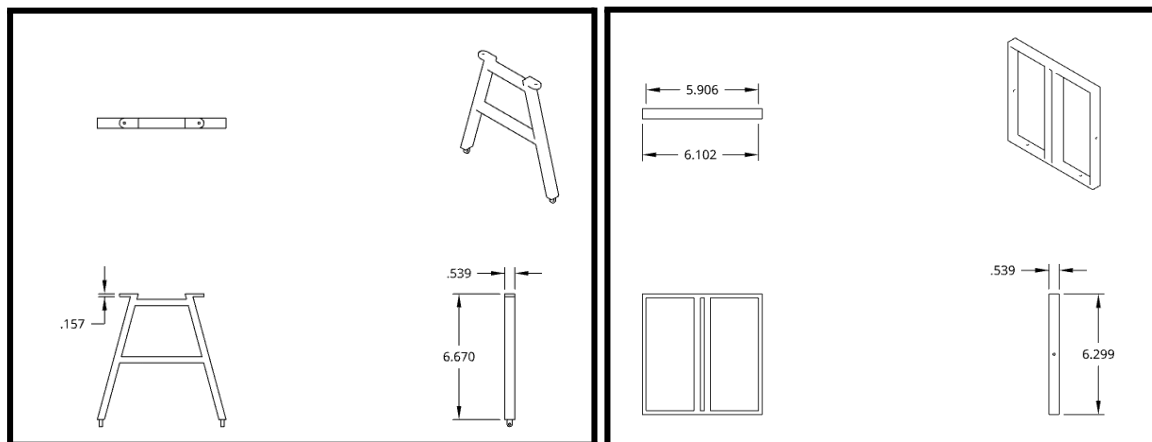


Figure 2 Support Arms and Sliding Carriages

Brackets / Connectors:

This small component was designed to enable the articulated connection between the backboard and the supports of the mechanism that drives the movement. It has a solid base with two holes for a secure and sturdy collar, and a double upper support with an axle that allows the lifting system to rotate and articulate smoothly and stably.

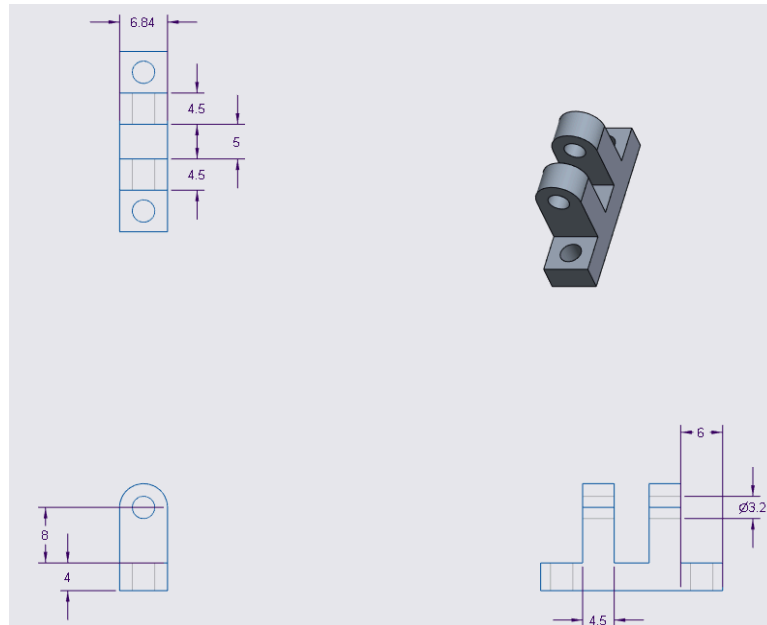


Figure 3 Brackets

Transmission Rod (Linkage / Connecting Rod):

It is the direct transmission between the stepper motor and the basket's moving structure. Its function is to convert the motor's motion into a vertical linear force that pushes the board support up or down, thereby generating the basket's regulated displacement.

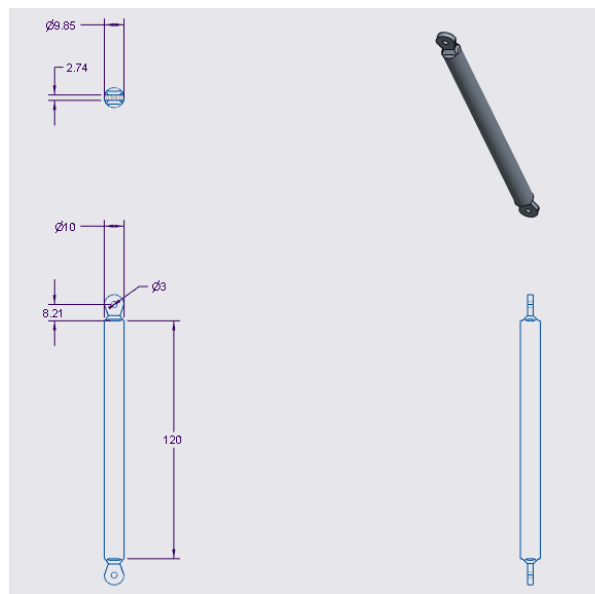
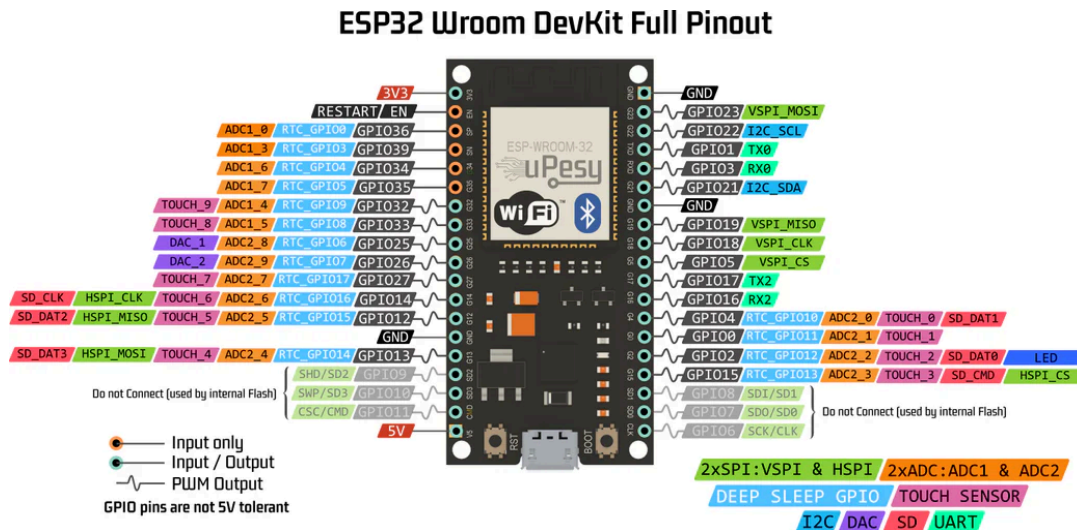


Figure 4 Transmission Rod

3.2.2. Electrical design

Connection Mapping and Pin Assignment

For the final assembly, we organized the ESP32 connections in a much more efficient manner (Table 1). Compared to the first version of the prototype, we had to reassign the pins of some components (such as the displays).



IR Basket Sensors

- **TOP Sensor (TSOP58438):** GPIO 15.
- **BOTTOM Sensor (TSOP58438):** GPIO 13.
- **IR Emitter LED (38kHz):** GPIO 12.

Displays (TM1637)

- **Display Timer CLK:** GPIO 26.
- **Display Timer DIO:** GPIO 4.
- **Display Score CLK:** GPIO 14.
- **Display Score DIO:** GPIO 27.

Basket Motor (NEMA 17)

- **STEP:** GPIO 32.
- **DIR:** GPIO 33.
- **ENABLE:** GPIO 25.
- **Limit Switch:** GPIO 35

Buttons and LEDs (Games)

- **Team A Button / Ready Button:** GPIO 16.
- **Team B Button:** GPIO 17.
- **Green LED (Pending basket / Indicator):** GPIO 21.
- **Blue LED (Team A scored):** GPIO 19.
- **Red LED (Team B scored):** GPIO 18.

Control Architecture (The “Brain”)

The core of the BasketBot electrical subsystem is governed by a central control unit based on the **ESP32 Wroom DevKit** microcontroller. This device acts as the "brain" of the smart basketball hoop, serving as the central node where data acquisition lines (sensors), game logic algorithms, mechanical actuation systems (motor), and the user interface (LEDs and web application) converge.

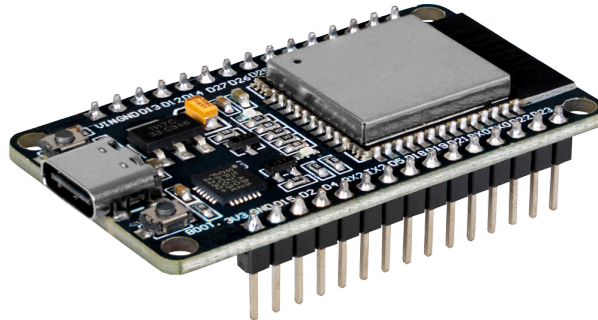


Figure 6 ESP32

The selection of the ESP32 instead of other market alternatives (such as 8-bit Arduino platforms) is based on the project's technical and connectivity requirements:

- **Integrated Connectivity:** The ESP32 incorporates certified hardware modules for Wi-Fi and Bluetooth (BR/EDR and BLE). This feature is essential to achieve the goal of connecting the basketball hoop to a web-based mobile application, eliminating the need for external communication chips that would increase both the cost and complexity of the electrical design.
- **High Peripheral Density and PWM Channels:** The chip provides a wide array of addressable GPIO pins, including internal timers, pulse generators (essential for the stepper motor driver), and serial communication ports.

Motor NEMA 17 stepper motor with integrated T8 lead screw

The selected motor is a NEMA 17 stepper motor with a 310 mm T8 lead screw integrated directly into the shaft. Its main electrical and mechanical characteristics are as follows:

Electrical Characteristics	
1. Number of Phase	2
2. Step Angle	1.8 °
3. Rated Voltage	3.3V DC
4. Rated Current	1.5 A
5. Holding Torque	400mN.m Min(2 Phases)
6. Phase Resistance	2.2 $\Omega \pm 10\%$ (20° C)
7. Phase Inductance	3.8mH $\pm 20\%$ (1kHz 1V rms)
8. Motor Inertia	57 g.cm ²
9. Motor Weight	280g Ref.
10. Insulation Resistance	100M Ω Min.(DC 500V)
11. Insulation Class	B(130° C)

Table 1 Electrical Characteristics

In order to meet the functional requirements, several technological alternatives were evaluated before making the final decision. The main selection criteria were: the ability to perform open-loop position control, meaning that it does not require an external encoder to determine its position; compatibility with the ESP32; and a cost that fits within the project's budget.

Sufficient torque to move the basket structure was also a critical factor. According to the manufacturer's datasheet, the chosen stepper motor is capable of lifting a maximum load of 12 kg. During the design phase, the total weight of our basket prototype was calculated to be approximately 6 kg. This means the motor offers twice the capacity required by the system, providing an ample safety margin to handle the structure effortlessly, prevent any loss of steps, and overcome unexpected friction during movement.

The main objective of this subsystem is to enable the motorized adjustment of the basketball hoop's height, allowing the BasketBot system to adapt to different types of players—whether children, teenagers, or adults—without requiring manual intervention.

The height adjustment is carried out through a lead screw mechanism driven by a stepper motor, which converts the motor's rotational motion into a vertical linear displacement of the structure supporting the basketball hoop.

The control of this subsystem is integrated directly with the project's mobile application. Through the app, the user can select the desired hoop height before starting a game or adjust it in real time using the up and down buttons. This command is transmitted via Wi-Fi

to the ESP32, which is responsible for generating the control signals required to drive the motor in the appropriate direction and reach the target position.

In addition, the subsystem incorporates two limit switches that physically define the maximum travel range of the mechanism, ensuring that the motor does not exceed the structural limits of the minimum and maximum permitted heights. This provides both system safety and protection for the mechanical components.

Driver: TMC2209

To control the NEMA 17 from the ESP32, a **TMC2209 driver** is used. This next-generation integrated circuit is specifically designed for bipolar stepper motors. It represents a significant improvement over more basic drivers such as the A4988, particularly in terms of quieter operation, smoother motion, and advanced functionality.

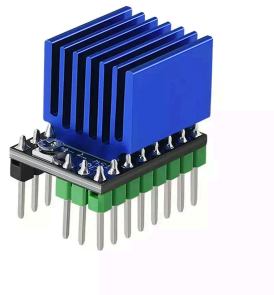


Figure 7 Driver TMC2209

Basic Operation of the Motor and Driver

The NEMA 17 is a bipolar stepper motor with two independent coils (Phase A and Phase B). The TMC2209 driver energizes these coils in a specific sequence, generating a rotating magnetic field that drives the rotor in precise, discrete steps with virtually silent operation.



Figure 8 Nema 17

Each time the ESP32 sends a rising edge to GPIO 32 (STEP pin), the TMC2209 advances the motor exactly 1.8° . The direction of rotation is determined by the logic level of GPIO 33 (DIR pin).

The TMC2209 driver internally manages the coil energizing sequence and current control, so the ESP32 only needs to generate two simple digital signals, without any additional motor control computations.

Digital Scoreboards (TM1637 Displays)

The BasketBot uses two 4-digit, 7-segment display modules controlled by the TM1637 chip (one for time and another for score).



Figure 9 TM1637

Its selection is justified by two key electrical reasons:

- **Massive pin saving:** Controlling a standard 4-digit display would require 12 wires connected to the ESP32. With two displays, this would amount to 24 GPIO pins, leaving insufficient connections for the rest of the system components. The TM1637 chip acts as an intelligent intermediary and only requires 2 data wires (CLK and DIO) per display, greatly saving space on the board.
- **Multiplexing (optical illusion):** The TM1637 does not power all digits simultaneously, as this would consume too much energy. Instead, it turns each digit on and off sequentially at an extremely high frequency (in the kHz range). Because this switching happens so quickly, the human eye does not perceive the flickering and instead sees all four digits lit at the same time, without overloading the system.

Display Connections

Since we are using two independent displays, each one requires a two-wire connection to receive data from the ESP32.

- **Time Display (Timer):** The clock line (CLK) is connected to **GPIO 26**, and the data to **GPIO 4**.
- **Score Display:** It remains connected as before, with CLK on **GPIO 14** and DIO on **GPIO 27**.

Feedback Indicators (RGB / Simple LEDs)

As a complement to the numeric score displays, a lighting system based on three visual indicators has been designed. These serve as a visual communication interface (Green LED, Blue LED, and Red LED).



Figure 10 LED

Their electrical configuration and activation logic correspond to the following stages of game interaction:

Green LED: At the exact moment the ball passes through the hoop successfully, the ESP32 toggles the pin connected to this LED, producing an instant green light that confirms a valid basket.

Team Assignment LEDs (Blue and Red): These are used to identify which of the two competing teams has scored points on the scoreboard. To prevent incorrect assignments, the software requires a prior validation step:

- **Physical Interface:** The system includes two manual push buttons (one for the blue team and one for the red team). Electrically, these buttons are configured as digital inputs with pull-up resistors to ensure clean and stable readings.
- **Digital Interface:** Alternatively, the assignment can be performed remotely through the web mobile application using the same control protocol.

LEDs: Each color is assigned its own digital output channel on the ESP32. The Green LED is connected to **GPIO 21**; the Blue LED (which confirms a point for Team A) is connected to **GPIO 19**; and the Red LED (which confirms a point for Team B) is connected to **GPIO 18**. Each wire includes a small series resistor to prevent excessive current from reaching the LED, which could otherwise damage either the LED itself or the microcontroller pin.

Buttons (Physical Push Buttons): Physically, we have two buttons, which helps us save components and board space. They work as follows:

- **1vs1 Game mode:** The Team A button is connected to GPIO 16 and the Team B button to GPIO 17. They are used to confirm which team made the basket that has just been scored.
- **Mode 21, Shootout and Free modes:** Not used.

Interconnection and Integration with Sensors

The system responsible for automatically and reliably detecting baskets uses a dual infrared (IR) optical barrier. The **TSOP58438** receiver was selected to avoid false positives caused by sunlight or artificial lighting, as it completely ignores surrounding ambient light and only responds to infrared bursts modulated precisely at 38 kHz.



Figure 11 TSOP58438

To ensure a stable electrical integration free from false detections, the pin-to-pin connection between the receiver and the control architecture has been designed as follows:

- **IR LED:** Connected to **GPIO 12**; it is the LED that emits the infrared light.
- **TOP Sensor:** Connected to **GPIO 15**. It acts as an input to notify the program at the exact moment the ball begins to enter the hoop.
- **BOTTOM Sensor:** Connected to **GPIO 13**. It acts as a second input to confirm that the ball has fully passed through and fallen completely.

3.2.3. Control and software

Game and Team Validation Flow Logic

To prevent errors or false scoring, a state machine has been implemented under a strict dual-condition flow:

1. **Detection Phase (Physical):** When the ball passes through the hoop, the IR sensors are activated. The ESP32 immediately turns on the Green LED to confirm that a valid basket has been physically detected. At this stage, no points are added to the scoreboard yet.
2. **Validation Phase (Human):** The system then waits for a few seconds. In order for the point to be registered, the scoring team must be indicated. This is done using the manual push buttons (one for the Blue team and one for the Red team) or directly through the web application.
3. **Update Phase:** Once the team signal is received (via button or Wi-Fi), the ESP32 turns off the Green LED, turns on the corresponding team LED (Blue or Red) as a visual confirmation, and automatically updates the Score Display (TM1637), adding the points to the appropriate team's score.

Game Modes

Free (Free Practice)

This module is a free training mode with no time or scoring limits, controlled by a single scoreboard.

Objective and operation:

- **Scoring:** Each valid basket detected by the sensor adds +2 points.
- **Visual effect:** When a basket is scored, a LED (GPIO 21) lights up for 200 ms. The function turns it off in the background without using blocking code.

Web Interaction

- **Manual control:** The user can add +2 points or use the "Undo" button to subtract 2 points in case of an error.
- **Safety:** The code uses $\max(0, _gl_puntos - 2)$ to prevent the score from going below zero.
- **Synchronization:** Data is sent in JSON format to update the mobile screen in real time.

Win Condition

There is no win condition. The game continues accumulating points until the user presses reset or stops it from the web interface.

The 21

This mode allows 1 to 4 players to play in an automated way. It controls an indicator LED to mark the shooting time and communicates with the web interface.

Objective and scoring

The objective is to reach exactly **21 points**. The game operates using a turn-based system based on a time window:

- **Normal Shot:** If the player scores, they earn +2 points and receive an extra “Free Throw” attempt. If they miss or time runs out, the turn passes to the next player.
- **Free Throw:** If the player scores, they earn +1 point and can continue taking free throws. If they miss, they lose their turn.

If the ball hits the rim and bounces out, the system immediately detects it as a miss and switches turns without waiting for the time to expire.

Win condition and penalty

- **Victory:** The first player to reach exactly 21 points wins.
- **Penalty:** If a player exceeds 21 points (for example, having 20 points and scoring 2), their score is automatically reduced to 11 points and they immediately lose their turn.

The 1 vs 1

This module manages a real-time match between two teams sharing the same hoop, using physical buttons to assign points. It uses GPIO 16 (Team A), GPIO 17 (Team B), and three control LEDs: Green (GPIO 21), Blue (GPIO 19), and Red (GPIO 18).

Objective and operation

- **Objective:** To score more points than the opponent before the game time runs out.
- **Validation window:** When a basket is scored, the Green LED turns on and a waiting state is activated. The first team to press their button within the time limit earns +2 points. If no button is pressed, the basket is cancelled.

End of match

When the timer reaches zero, the function turns off all LEDs, locks the game, determines the winner (or a draw), and sends the final result to the web interface.

Shootout

This module manages a timed competitive turn-based mode (1 to 4 players). It uses a “Ready” button (GPIO 16) and an indicator LED (GPIO 21).

Objective and operation

- **Objective:** Score as many baskets as possible within a time limit (e.g., 30 seconds). Each successful shot adds +1 point.
- **Asynchronous timing window:** If the game is paused from the web interface, the code calculates the time spent paused and compensates for it by adding it back to the board's timer, ensuring the player does not lose any seconds.

Win condition and tiebreaker

- **Victory:** The player with the highest score wins.
- **Automatic tiebreaker:** If there is a tie for first place, an exclusive group is created where only the tied players compete. Their scores are reset to 0 for an additional round. This process is repeated until a single winner is determined.

Motor Control

To ensure everything works correctly and without freezing, the programming has been structured as a state machine that runs asynchronously (non-blocking). This is essential so that the ESP32 doesn't get "frozen" while the motor is moving; it allows the microcontroller to continue performing other tasks simultaneously, such as updating the scoreboard or managing communications. Instead of making long, single-shot movements, the code uses a function called `motor_tick()` that runs continuously in the program's main loop. Every time this function is called, the ESP32 checks the current state of the system and, if necessary, moves the motor a single step before continuing with the rest of the code.

The main states managing this logic are the following:

- **Idle** (*MC_IDLE*): The motor has reached its destination and remains stopped, waiting for a new height command.
- **Homing** (*MC_HOMING_DOWN* and *MC_HOMING_UP*): These are the two phases to find the zero point using the limit switch. First, the motor moves down until it triggers the limit switch, and then it moves up a few steps to release it, establishing this position as the reference origin.
- **Moving** (*MC_MOVING*): The active state where the motor moves (upwards or downwards) to the height in centimeters requested by the user. To do this, the system converts the desired height into a number of motor steps by applying a conversion factor of 2,000 steps per centimeter, derived from the pitch of the T8 lead screw and the motor's step angle.
- **Error** (*MC_ERROR*): The failure state the system enters when a critical anomaly is detected. In this case, the motor is immediately disabled to protect the mechanical structure.

Finally, essential safety measures have been included within the code itself to protect the mechanical structure. On one hand, if the motor moves down during the initial homing phase without finding the limit switch, the system assumes a fault has occurred, stops immediately, and enters the *MC_ERROR* state, disconnecting the motor to prevent any damage. On the other hand, if the motor is moving normally and unexpectedly hits the limit switch, the software instantly cancels the movement and resets the position to zero to avoid mechanical collisions.

Hoop Sensors

Phase 1: Testing and calibration program

Before integrating the sensors with the web interface or the motors, we created a simple standalone program solely to test how the IR receivers responded. This code continuously monitored the pins and displayed a message on the computer screen every time the ball passed through. Thanks to this test program, we were able to calibrate the infrared light distance and verify that the 38 kHz modulation of the emitting LED worked even under ambient light conditions.

Phase 2: Final Code

The final code was designed around three technical pillars:

Asynchronous operation: Instead of using typical delays, the code calculates real elapsed time using the ESP32's internal clock. This is essential so the sensor can monitor the ball in the background while, at the same time, the hoop motor is moving or the user is interacting with the web application.

IR burst logic: The LED is not kept continuously on. The program activates the emitter in bursts, turns the LED off, and at that precise moment performs the digital reading. It then leaves a recovery margin before the next cycle.

State machine flow control: The program manages the ball trajectory sequentially to avoid scoring errors:

- **Waiting:** It waits for the upper sensor to be triggered. When this happens, it records the time and opens a safety window.
- **Checking:** If the lower sensor is triggered within 600 ms, the basket is validated. If the time limit is exceeded, the code assumes the ball has bounced out and returns to the initial state without scoring.
- **Anti-bounce:** Once a basket is detected, the system locks for 1 second. During this period, sensors are ignored to prevent rim vibrations from generating extra points.

Displays

Score

Depending on the game mode, the score display behaves differently:

- **1vs1 mode:** The display is split into two halves of 2 digits each. The left side shows Team A's score and the right side shows Team B's score (up to 99). The middle segments are disabled to simulate two independent scoreboards.
- **21, Shootout and Free modes:** The display acts as a single 4-digit counter (0000–9999), showing the total score or the score of the active player's turn.

Whenever a match is paused, has ended, or the system is in standby, the display shows four fast dashes (----) to visually indicate that the counter is inactive.

Timer

The same applies to the timer display:

- **1vs1 mode:** Displays the total game time in minutes and seconds (MM:SS). To achieve this, the code calculates the quotient and remainder of the remaining time.
- **Shootout and 21 modes:** Since these are fast countdown-based shooting modes, the display is configured in integer seconds (SS), using leading zero padding for proper visual alignment.
- **Free mode:** Since this mode has no time restriction, the timer is disabled.
- **Inactive state:** As with the scoring module, if the game is paused, finished, or in standby, it shows four fast dashes (----) as a visual indicator that the game is frozen.

Web Application and Wi-Fi Communication

This subsystem interconnects the mobile web application with the ESP32 control logic through an asynchronous architecture, ensuring an instantaneous response (low latency) and completely independent operation from the internet or external network infrastructures.

- **Access Point Mode:** The ESP32 generates its own local Wi-Fi network with a dedicated SSID. Users connect directly to this network and access the control interface by entering the microcontroller's local IP address into any mobile web browser, eliminating the need for external routers or servers.
- **Control Interface & Features:** The web application features a responsive design optimized for smartphones. Through this graphical interface, users can select from the four available game modes (Free Practice, The 21, 1 vs 1, and Shootout), adjust the hoop's mechanical height in real-time, and manage the scoreboard with specialized correction features such as the Undo button.
- **Asynchronous Data Flow (JSON):** Data exchange between the control board and the mobile interface is handled via structured JSON payloads. By implementing non-blocking asynchronous programming based on the ESP32's internal clock (avoiding blocking code like `delay()`), the mobile screen updates instantaneously whenever a basket is registered or a timer changes, without ever interrupting background sensor polling or stepper motor movements.

3.3. Functional testing of the prototype

To validate the performance of the BasketBot, a series of experimental tests were conducted:

1. **Height Adjustment Test:** Initially, although the motor had sufficient torque according to its datasheet and responded perfectly to commands, the mechanism failed to lift. We discovered the issue was a lack of structural stability: the weight of the hoop tilted the motor's support shaft, preventing movement since the system requires perfect vertical alignment to avoid jamming. To solve this, we added a counterweight to balance the load and reinforcing bars to keep the structure completely upright. Thanks to these modifications, the lifting mechanism now functions perfectly, smoothly, and stably.
2. **Score Detection Accuracy:** 100 shots were taken to evaluate the detection system's accuracy, obtaining 87 correct detections (87%). During testing, it was found that the upper sensor required readjustment to improve detection, so it was temporarily disabled until the final version.

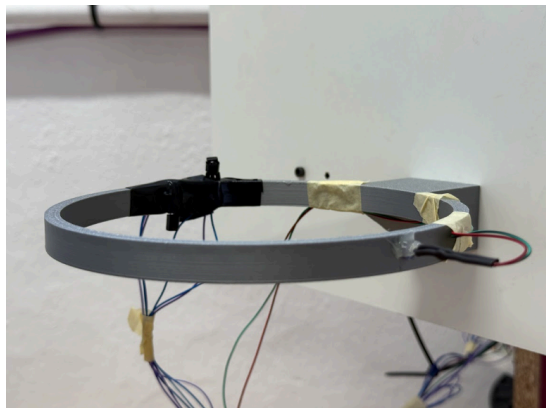


Figure 12 Score Detection

3. **App Synchronization and Latency:** The synchronization between the mobile application and the physical system was highly successful and operated without any major complications. Across all available features and game modes, the application detected and updated the scoreboard with a latency that was practically unnoticeable to the user.

Furthermore, regarding the response time when selecting different options or changing settings within the app, the system delivered excellent results, executing commands almost instantaneously. Overall, the data transmission proved to be highly reliable, stable, and free of any unexpected disconnections.

4. **Limitations Identified:** Throughout the development of the project, we identified two primary limitations that affected the system's initial performance:
 - Lifting Mechanism: As previously mentioned, a lack of structural stability and a slight tilting of the main support shaft initially prevented the motor from

moving, an issue we successfully resolved by adding a counterweight and reinforcing bars.

- Sensor System: We encountered several issues regarding the synchronization of the detection sensors. We could not get them to function simultaneously; whenever one sensor was connected, the other stopped working. Troubleshooting this specific conflict and fixing the connection errors required a significant amount of time and testing.

4. Cost of the project

This section outlines the total economic investment required for the development of the BasketBot prototype. The budget is divided into human resource expenses (man-hours) and the physical material component procurement.

4.1. Engineering Man-Hours

The project development was collaboratively executed over a 15-week timeline by the four team members. A standard engineering labor rate of 20.00 €/hour has been applied to quantify development costs. The labor distribution across the mechatronic engineering areas is detailed in the table below:

Team Member	Project Role	Total Hours	Labor Cost (€)
Èric Vila	Project Manager	106	2120 €
Ferran Soler	Mechanical Leader	100	2000 €
David Sánchez	Electronics Leader	99	1980 €
Biel Cruz	Software Leader	75	1500 €
		Total:	7600 €

Table 2 Man-Hours

4.2. Material Budget

The physical component expenses correspond directly to the preliminary shopping list managed for prototyping. The prototype remains well within the university-established 300.00 € threshold:

Material	Quantity	Unit Price (€)	Total Cost (€)
Motor NEMA 17 Stepper	1	32.22	32.22
Driver TMC2209	1	10.32	10.32
ESP32	1	6.87	6.87
TSOP4838	1	11.04	11.04
TM1637	2	5.6	11.2
Push button	2	0.3	0.6
Limit sensor switch	1	1	1
Rsistor 100 Ω	1	1	1
Rsistor 330 Ω	3	1	3
LED IR de 940 nm	1	5.99	5.99
Blue Led	1	0.3	0.3
Green Led	1	0.3	0.3
Red Led	1	0.3	0.3
Protoboard	1	2	2
		Total:	86.14 €

Table 3 Material Budget

4.3. Total Project Budget

The total financial execution sum of the BasketBot mechatronic development combines both direct material acquisition and calculated engineering engineering assets:

Total Project Cost = 7600 + 86.14 = 7686.14€

The resulting financial distribution demonstrates that while the physical components and prototyping materials were kept well under the university-established limits to ensure affordability, the true value of the mechatronic asset lies in the engineering man-hours dedicated to system integration, software namespacing architecture, and structural hardware optimization.

5. Conclusions

BasketBot was born from a situation we've all experienced at some point: you're playing, the game heats up, and suddenly nobody agrees on the score. That classic argument of "was it 10 or 11?" is exactly the problem we set out to solve. And we believe we've succeeded.

Throughout this semester, we have designed, programmed, and built a complete mechatronic system that automates the scoreboard, manages turns, and controls game time, all integrated into a web application accessible from a smartphone without the need for an external internet connection. The ESP32 has proven to be the ideal heart of the system, simultaneously coordinating basket detection, motor control, scoreboard updates, and Wi-Fi communication without any of these tasks interfering with one another. In all the tests conducted, the application's response has been practically instantaneous, and data transmission has remained stable and reliable across all game modes.

The lifting mechanism, one of the project's most demanding mechanical challenges, ultimately functioned smoothly and stably after we resolved a structural alignment issue that initially prevented movement.

However, detection accuracy reached 87% over 100 real shots, and the upper sensor had to be temporarily disabled due to a synchronization conflict between the two infrared receivers. These are real limitations that we aren't hiding; rather, they clearly outline the path for future improvements: resolving the sensor interference to exceed 95% accuracy, permanently soldering the connections on the protoboard to improve system robustness, and adding closed-loop control to the lifting system.

Ultimately, BasketBot is a system designed for everyone and built for fun. It lets the hoop do the work so players can focus on what really matters: the game.

6. Bibliography

Hardware Datasheets & Technical Documentation

- [1] Espressif Systems, ESP32-WROOM-32 Datasheet, Espressif Systems Hardware Documentation. <https://www.espressif.com/en/support/documents/technical-documents>
- [2] TMC2209 Datasheet: Ultra-silent driver for two-phase stepper motors, https://www.alldatasheet.com/view.jsp?Searchword=Tmc2209%20datasheet&gad_source=1&gad_campaignid=1437346752&gbraid=0AAAAADcdDU9n3MsT7uXORuqmnVsRsR1ZS&clid=Cj0KCQjw0JnRBhDJARIsALobnXYkqYNrMtxQESbwQrHn_2EVLDrZmEJRADI0vlgcm eMot0xV9XpYmsaAsU-EALw_wcB
- [3] Titan Micro Electronics, TM1637 LED Drive Control Special Circuit Datasheet. <https://robotlandia.es/display/848-pantalla-de-4-digitos-y-7-segmentos-con-driver-tm1637.html>
- [4] Vishay Semiconductors, TSOP4838 / TSOP58438 IR Receiver Modules for Remote Control Systems, Vishay Intertechnology. <https://www.vishay.com/>
- [5] Texas Instruments, LM2596 SIMPLE SWITCHER Power Converter 150-kHz 3-A Step-Down Voltage Regulator, Texas Instruments Technical Documentation. <https://www.ti.com/product/LM2596>
- [6] NEMA 17 Stepper Motor: Manufacturer's specifications for the model with the integrated T8 lead screw

Software & Development Tools

- [7] Arduino, Arduino IDE Documentation and Language Reference.
- [8] GitHub Docs, Git and GitHub Documentation, GitHub. <https://github.com/Integrated-Project-2-2026-UVic-UCC/BasketBot-T2>